Centralina di controllo per ASG

Introduzione

La centralina di controllo per ASG (AirSoft Gun) è una scheda da montare sopra i fucili da softair che permette di contare i proiettili sparati, controllare lo stato della batteria e tenere in memoria il numero dei proiettili utilizzati nelle ultime cinque partite.

Dato che per alimentare i fucili si utilizzano delle batterie LiPo è importante conoscere costantemente la percentuale di carica residua, la tensione totale, la tensione delle singole celle e la differenza di tensione delle celle.

Per compiere queste operazioni sono stati inseriti nel circuito due partitori di tensione (descritti al paragrafo "Lettura della tensione") che abbassano le tensioni della batteria a livelli consoni con la logica di funzionamento del microcontrollore.

Tramite l'algoritmo sviluppato è possibile visualizzare sul display OLED la percentuale di batteria residua, la tensione totale della batteria e la tensione della cella 1 e della cella 2.

Per facilitare l'utilizzo della scheda da parte dell'utente il sistema è stato programmato in modo da controllare che la tensione della batteria non scenda sotto una soglia critica o che le due celle non si scarichino in maniera differente.

In caso si verifichi un errore comparirà una scritta che avvertirà l'utente del tipo di problema.

In caso di problemi multipli compariranno più messaggi d'errore.

Per contare i proiettili sparati è stato progettato un sistema composto da fotodiodo IR e LED IR, un comparatore di tensione, un potenziometro per regolare la sensibilità e un pin del microcontrollore abilitato per la tecnica dell'interrupt (descritto nel paragrafo "Rilevazione proiettili").

Il sistema prevede una memoria permanente (EEPROM) che memorizza i colpi sparati nelle ultime cinque partite (descritto nel paragrafo "EEPROM, salvataggio partite").

Inoltre è previsto il salvataggio nella EEPROM anche dei valori settati dall'utente nel menu impostazioni (descritto nel paragrafo "EEPROM, salvataggio impostazioni").

Viene fornita l'opportunità all'utente di accedere ad un menu che permette di modificare l'intervallo di tempo con cui viene letta la tensione, l'intervallo di tempo per il refresh del display e l'intervallo di tempo con cui i dati vengono salvati nella EEPROM (descritto nel paragrafo "Display e menu").

Per aumentare la versatilità della scheda è stato usato il protocollo I2C per comunicare con il display.

In questo modo il display OLED può essere sostituito con un display LCD.

Nel caso avvenisse questo cambiamento sarebbe necessario modificare le librerie utilizzate per la comunicazione, inoltre è necessario controllare che la corrente assorbita non ecceda quella massima erogabile dal LDO e che la potenza dissipata da quest'ultimo non sia eccessiva.

Per programmare il microcontrollore è stato utilizzato l'ambiente di sviluppo di Arduino perché offre un sistema compatibile con il micro e una buona compatibilità con le librerie utilizzate.

Nel PCB sono stati esposti i pin necessari per la programmazione.

In questo caso specifico per la programmazione del microcontrollore è stato utilizzato un Arduino Mega in configurazione di ISP (In System Programming).

Lo sketch che permette di caricare il codice su un microcontrollore utilizzando un Arduino Mega è stato fornito dagli sviluppatori della piattaforma Arduino.

Nel caso non si volesse utilizzare un Arduino per programmare la scheda è possibile utilizzare un programmatore FTDI che permette di interfacciare il microcontrollore con il computer tramite un cavo USB.

Come microcontrollore è stato utilizzato un ATtiny84 prodotto dalla Microchip.

È stato scelto questo microcontrollore perché ha tutte le funzioni necessarie per essere utilizzato all'interno del circuito.

Presenta una memoria programma di 8192 byte e una memoria variabili di 512 byte.

Il programma scritto per il sistema utilizza 8046 byte di memoria programma e 441 byte di memoria variabili.

Progettazione

Per disegnare lo schematico ed il PCB è stato utilizzato il software EasyEDA.

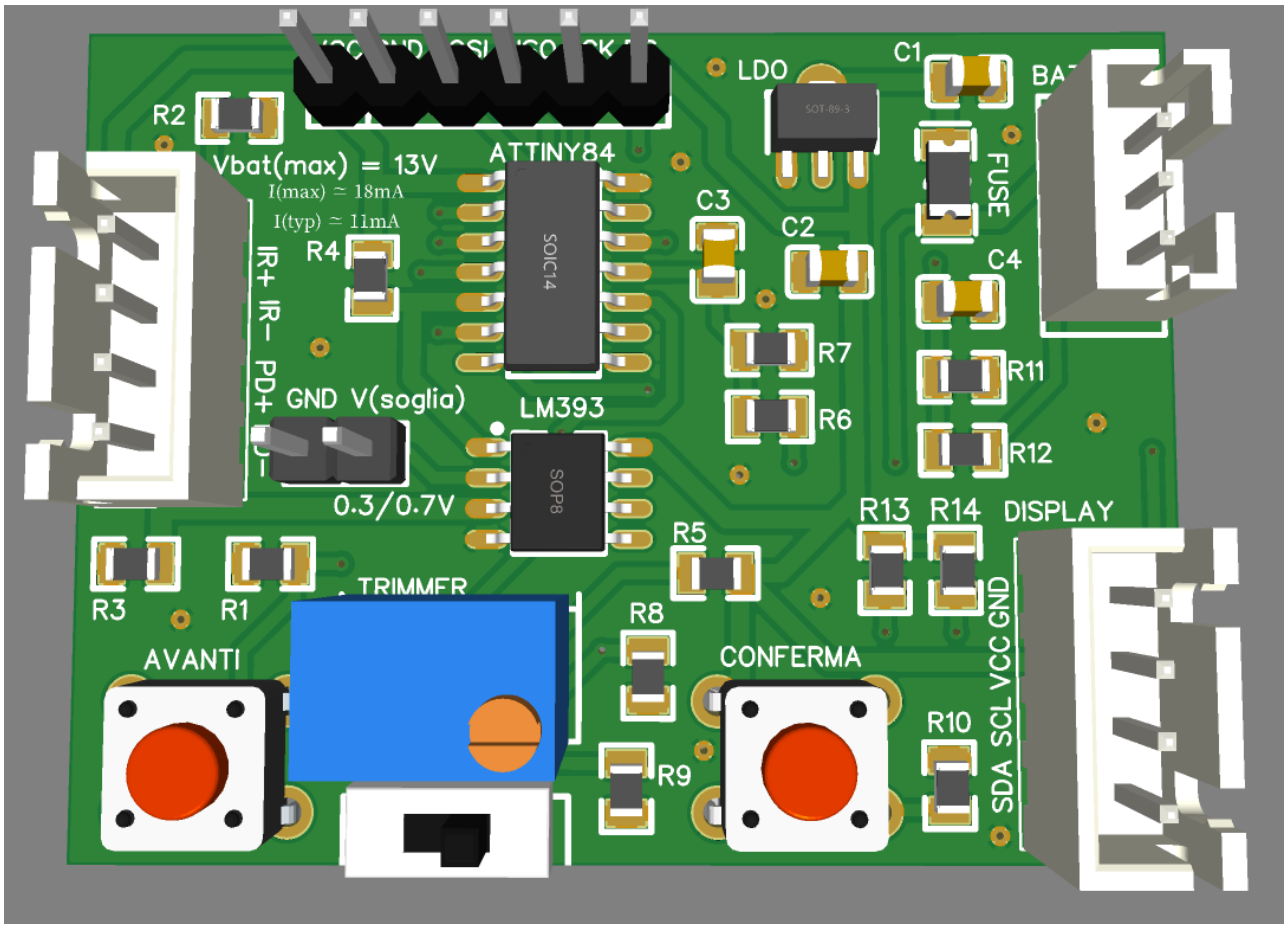
I componenti sono stati scelti della tipologia SMD per diminuire la grandezza dell'intera scheda in modo da poterla installare più facilmente sopra un fucile.

Le piste di alimentazione e la pista del segnale di interrupt hanno una larghezza di 40 mils (1mm circa) mentre le piste di segnale sono di 20 mils (0.5mm).

La pista che porta il segnale di interrupt è stata fatta da 40 mils per limitare il più possibile gli effetti dell'impedenza che si viene a creare.

Per evitare che le piste di massa creassero dei loop sono stati inseriti due piani di massa, uno su ogni faccia della scheda.

Durante la fase di progettazione per verificare la corretta disposizione dei componenti sopra il PCB è stato utilizzato il tool "3D view" presente all'interno del software che permette di simulare come sarà la scheda una volta montati tutti i componenti.



Per dimensionare correttamente i componenti utilizzati nel circuito sono state utilizzate le formule di progetto fornite all'interno dei datasheet e formule basilari come la legge di Ohm.

Una volta effettuati tutti i calcoli si è proceduto all'acquisto dei componenti.

Come per ogni progetto alcuni valori teorici differiscono da quelli reali che si trovano in commercio, di conseguenza sono stati acquistati componenti che avessero un valore il più simile possibile a quello teorico.

Soprattutto per le resistenze dei due partitori i valori si discostano leggermente da quelli trovati empiricamente.

Non è stato possibile acquistare i valori esatti, anche se presenti, perché avrebbero avuto un package differente dallo 0805.

Di conseguenza sono stati inseriti ed acquistati componenti che rispettassero il package e avessero i valori più simili possibile.

È stato utilizzato un ATtiny84 perché è un microcontrollore piccolo e versatile dotato di tutte le specifiche necessarie per la realizzazione della scheda.

Per rilevare il passaggio dei proiettili è stato usato un comparatore esterno perché presenta un tempo di risposta molto basso che risulta essere essenziale per il corretto conteggio dei colpi sparati dal fucile.

Il comparatore scelto è un LM393 e possiede un'uscita di tipo "open collector", di conseguenza è stato necessario inserire una resistenza di pull-up che permetesse di definire due stati stabili del pin di interrupt.

Dato che il pin di interrupt risulta essere normalmente alto e solamente una volta che il comparatore rileva un proiettile commuta allo stato logico basso l'interrupt è stato programmato in modo che conti esclusivamente sul fronte di discesa.

In questo modo si evita che se per qualche motivo il pin rimane ad uno stato logico basso per troppo tempo il sistema conti dei pallini che in realtà non sono stati sparati.

È stato scelto di far lavorare tutto il circuito in logica 3.3V perché da datasheet il microcontrollore, funzionando con un oscillatore interno a 8MHz, necessita di un'alimentazione da 3.3V anziché da 5V.

Per regolare la tensione dai 7.4V nominali della batteria ai 3.3V richiesti dal circuito è stato utilizzato un regolatore con bassa caduta di tensione (LDO – Low Dropout Regulator).

Il regolatore è un MCP1702T-3302E/MB prodotto dalla Microchip.

Questo regolatore permette di avere in uscita una tensione stabile di 3.3V con un errore del $\pm 0{,}4\,\%$.

Accetta una tensione in ingresso massima di 13.2V ed eroga una corrente massima di 250mA.

Da datasheet la potenza massima dissipabile dal dispositivo a causa del package è di 0,554W.

La potenza dissipata dal regolatore si calcola con la seguente formula $P = \left(V_{\text{in}} - V_{out}\right) * I$.

Quanto la tensione d'ingresso è uguale alla tensione nominale della batteria la corrente massima erogabile è di $I = \frac{P_{max}}{V_{\text{in}} - V_{out}} = \frac{554\,mW}{7.4\,V - 3.3\,V} = 135{,}12\,mA$.

Nel caso in cui la tensione d'ingresso sia uguale alla tensione massima del regolatore la corrente disponibile sarebbe uguale a $I = \frac{P_{max}}{V_{\text{in}} - V_{out}} = \frac{554\,mW}{13.2\,V - 3.3\,V} = 55{,}96\,mA$.

Presenta un fattore tipico di riduzione del ripple (PSRR – Power Supply Rejection Ratio) di 44dB.

Questo vuol dire che si ha una riduzione del rumore sulla tensione in ingresso pari a $10^{\frac{44}{20}} = 158{,}49$.

Se in ingresso è presente un disturbo di 10mVpp sull'uscita sarà presente lo stesso disturbo con una tensione picco-picco di soli 63,1µV.

I condensatori C1 e C2 di valore 1µF sono stati posizionati e selezionati seguendo le indicazioni del datasheet.

Nonostante il regolatore sia provvisto di un sistema di protezione dai cortocircuiti è stato inserito un fusibile per ovviare a possibili malfunzionamenti della protezione.

Il regolatore presenta anche una protezione per le alte temperature, in questo modo quando dissipa troppa potenza si disattiva per evitare guasti.

Non è stato necessario inserire un diodo di protezione per l'inversione di polarità della batteria perché sono stati usati dei connettori JST.

Grazie alla loro forma questi particolari connettori prevengono un eventuale montaggio al contrario.

Sono stati inseriti due pin header per misurare la tensione presente sul pin invertente del comparatore.

In questo modo risulta più facile controllare l'andamento della tensione mentre si effettuano correzioni.

Diagrammi di flusso

I diagrammi di flusso sono stati inseriti alla fine della relazione perché inserendoli all'interno delle pagine l'immagine sarebbe stata compressa e le scritte non sarebbero più state leggibili.

Una volta alimentato, il sistema entra nella fase di Setup.

Per prima cosa vengono settati i registri utilizzati per attivare o regolare le varie periferiche presenti all'interno del microcontrollore.

I registri utilizzati sono: GIMSK, MCUCR, ADMUX e ADCSRA.

Le funzioni di questi registri verranno spiegate nei paragrafi successivi.

Successivamente vengono definiti gli stati dei pin utilizzati.

La maggior parte dei pin sono settati come ingressi mentre i pin utilizzati per la comunicazione I2C vengono settati dalla libreria utilizzata.

Nella terza fase il sistema recupera tutti i dati salvati nella EEPROM come il numero di proiettili sparati nelle utlime partite.

Finito il setup si entra nel Loop.

Dentro il Loop se il selettore orizzontale è nella prima posizione (Selezione1) il sistema entra nel primo ciclo if.

All'interno di questo ciclo se viene premuto il tasto "Avanti" il sistema mostra la schermata con i colpi sparati nelle ultime cinque partite.

Per tornare alla schermata principale è necessario premere il tasto "Conferma".

Tramite l'utilizzo di "Millis" si contolla il tempo di refresh dello schermo.

Nel caso sia passato un tempo maggiore della variabile "Tempodisplay" viene aggiornato il display.

Nel caso il selettore orizzontale sia nella seconda posizione (Selezione2) si passa nel secondo cislo if utilizzato per modificare le impostazioni.

All'interno di questo ciclo viene mostrato il menu impostazioni.

Premendo il tasto "Avanti" viene incrementato la variabile "m" utilizzata per selezionare l'impostazione da modificare.

Se viene premuto il tasto "Conferma" il sistema incrementa l'indice relativo all'impostazione selezionata.

Questo processo viene spiegato nel dettaglio nel paragrafo "Display e menu".

Al di fuori di questi cicli sono presenti due if che permettono la temporizzazione della lettura della tensione e del salvataggio dei dati all'interno della EEPROM.

Viene utilizzata la funzione "Millis", come per il refresh del display, per evitare di interrompere il programma.

Nel caso si fosse utilizzata la funzione "delay" per creare le temporizzazioni il programma sarebbe stato bloccato, di conseguenza non avrebbe funzionato bene.

Lettura della tensione

Per leggere la tensione della batteria è stato utilizzato un ADC di tipo SAR (ADC ad approssimazioni successive) con risoluzione di 10 bit.

La batteria utilizzata per alimentare il circuito presenta una tensione nominale di 7,4V.

Questa tensione deriva dalla somma delle tensioni nominali delle due celle che compongono la batteria.

Per evitare di compromettere l'integrità della batteria è bene accertarsi che la sua tensione non scenda sotto una soglia critica.

Nel caso scendesse sotto questa soglia potrebbero verificarsi danni irreparabili alla capacità della batteria o all'integrità delle celle causandone una possibile combustione nella fase di ricarica successiva.

Nel caso specifico di questa batteria, essendo composta da due celle in serie, la tensione totale non deve scendere al di sotto di 6.4V (soglia critica).

La tensione minima della batteria è calcolata in base al numero di celle che la compongono.

Leggendo le raccomandazioni delle case costruttrici si legge che la tensione della singola cella non deve mai scendere al di sotto di 3,2V di conseguenza, avendone due in serie, la tensione minima sarà di 6,4V.

Per leggere le varie tensioni presenti nel circuito è stato utilizzato l'ADC presente all'interno del microcontrollore.

Dato che all'interno del microcontrollore è presente un solo ADC per leggere i vari canali analogici il costruttore ha previsto un MUX (Multiplexer) che ciclicamente collega i vari canali con l'ADC.

Se nel circuito non ci fosse un MUX sarebbe necessario avere un ADC per ogni canale analogico.

Il circuito funziona in logica 3.3V di conseguenza sono stati dimensionati due partitori resistivi che permettessero di abbassare le due tensioni che si vogliono misurare (8.4V e 4.2V) sotto la soglia della Vcc.

Se non si fosse abbassata la tensione l'ADC non avrebbe lavorato correttamente.

Di seguito si riportano i calcoli effettuati per dimensionare correttamente le resistenze.

Partitore 1

$$2.5\,V = \frac{R_2}{R_1 + R_2} * 8.4\,V \;\rightarrow\; 2.5\,R_1 + 2.5\,R_2 = 8.4\,R_2 \rightarrow 2.5\,R_1 = 5.9\,R_2 \rightarrow\; R_1 = \frac{5.9}{2.5} R_2$$

I valori delle resistenze utilizzate sono: R1 = 62kΩ e R2 = 27kΩ.

Partitore 2

$$2.5\,V = \frac{R_2}{R_1 + R_2} * 4.2\,V \;\rightarrow\; 2.5\,R_1 + 2.5\,R_2 = 4.2\,R_2 \rightarrow 2.5\,R_1 = 1.7\,R_2 \rightarrow\; R_1 = \frac{1.7}{2.5} R_2$$

I valori delle resistenze utilizzate sono: R1 = 16kΩ e R2 = 27kΩ.

La tensione d'uscita dei partitori durante lo svolgimento del dimensionamento è stata impostata a 2.5V per permettere di variare i valori delle resistenze nel caso non si trovassero in commercio resistenze con valori uguali a quelli calcolati.

Tutte le resistenze sono state scelte per una potenza massima di 1/4W e in package 0805.

Data l'impossibilità nel reperimento di resistenze con valori identici a quelli calcolati sono state acquistate resistenze con valori il più vicino possibile a quelli riscontrati in teoria.

Per rendere più costante la tensione in uscita dal partitore sono stati usati due condensatori ceramici da 100nF.

I dati di tensione che il sistema fornisce sono la media di quattro valori acquisiti ad intervalli regolari di dieci secondi.

Di conseguenza la tensione della batteria sarà aggiornata ogni quaranta secondi.

Durante la fase di test del software è stato notato che la lettura della tensione non era precisa.

Per limitare questo problema sono stati acquisiti dei valori ad intervalli regolari nel corso della giornata ed è stato fatto un grafico per poter visionare la curva di tensione letta dall'ADC rispetto a quella reale letta dal multimetro.

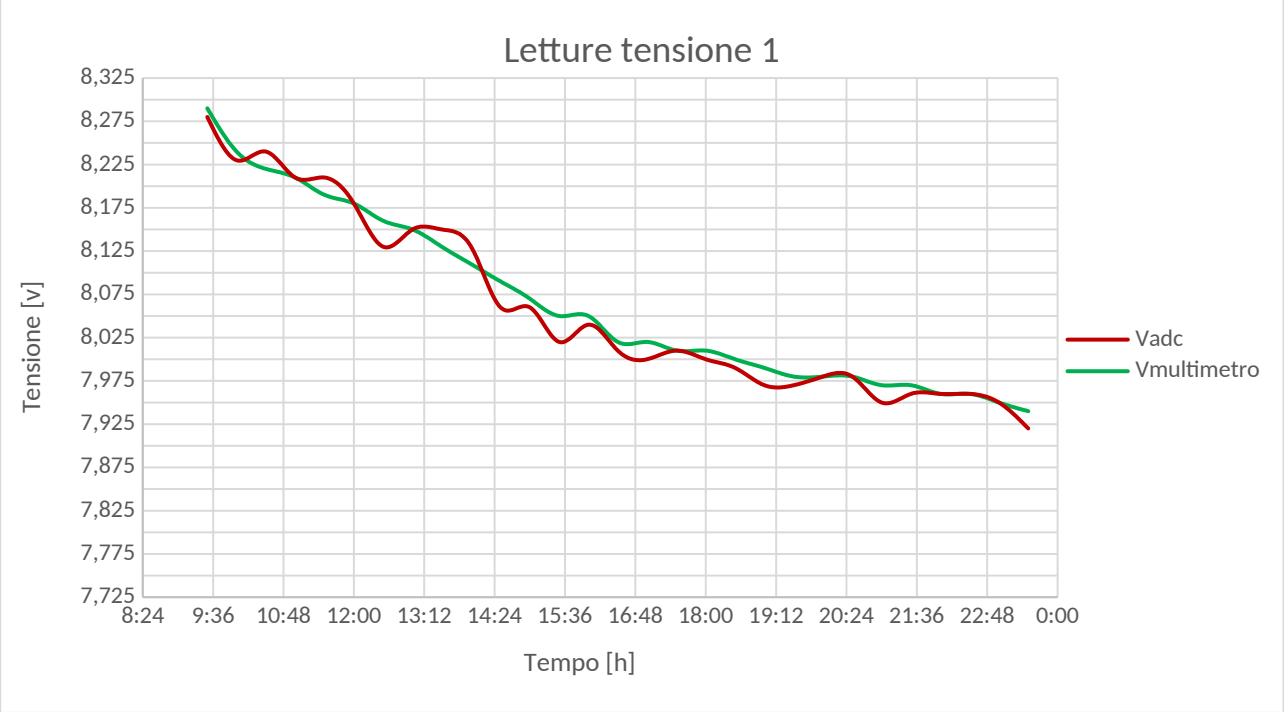
Il primo grafico e la prima tabella riportano i dati presi con la versione 1 del software.

Idealmente la curva rossa (tensione letta) e la verde (tensione reale) dovrebbero essere sovrapponibili.

Come si nota nel primo grafico la tensione letta dall'ADC non è quasi mai uguale a quella reale.

La differenza massima registrata è stata di 20mV e, nel periodo complessivo di quattordici ore, l'errore di tensione è di 12mV.

Dato che i risultati non erano soddisfacenti sono stati eseguiti ulteriori test modificando e ricalibrando di volta in volta il software.



Letture tensione 1				
Vadc [V]	**Vmultimetro [V]**	**Tempo [h]**	**errore [V]**	**errore [%]**
8,28	8,29	9:30	0,010	0,121

8,23	8,24	10:00	0,010	0,121
8,24	8,22	10:30	0,020	0,243
8,21	8,21	11:00	0,000	0,000
8,21	8,19	11:30	0,020	0,244
8,18	8,18	12:00	0,000	0,000
8,13	8,16	12:30	0,030	0,368
8,15	8,15	13:00	0,000	0,000
8,15	8,13	13:30	0,020	0,246
8,13	8,11	14:00	0,020	0,247
8,06	8,09	14:30	0,030	0,371
8,06	8,07	15:00	0,010	0,124
8,02	8,05	15:30	0,030	0,373
8,04	8,05	16:00	0,010	0,124
8,01	8,02	16:30	0,010	0,125
8,00	8,02	17:00	0,020	0,249
8,01	8,01	17:30	0,000	0,000
8,00	8,01	18:00	0,010	0,125
7,99	8,00	18:30	0,010	0,125
7,97	7,99	19:00	0,020	0,250
7,97	7,98	19:30	0,010	0,125
7,98	7,98	20:00	0,000	0,000
7,98	7,98	20:30	0,000	0,000
7,95	7,97	21:00	0,020	0,251
7,96	7,97	21:30	0,010	0,125
7,96	7,96	22:00	0,000	0,000
7,96	7,96	22:30	0,000	0,000
7,95	7,95	23:00	0,000	0,000
7,92	7,94	23:30	0,020	0,252
		Totale	0,012	0,150

Una volta creato il primo grafico, aver effettuato delle ricerche nei forum e nel datasheet del microprocessore sono state apportate delle modifiche alla parte dell'algoritmo che gestisce le misure di tensione per ottenere delle letture più precise.

Per prima cosa è stata modificata la formula in modo tale da avere una maggiore precisione nella conversione da dato analogico a numero di tensione.

Successivamente è stato inserito un prescaler che influisce sul clock dell'ADC.

Il prescaler inizialmente non era inserito ma nel datasheet si trova scritto che il clock dell'ADC dovrebbe rientrare entro certe frequenze per ottenere una buona precisione, dato che senza si eccede il limite è stato necessario inserirlo.

Inoltre tramite un registro apposito è stata settata la tensione di alimentazione come tensione di riferimento (3.3V) per eseguire le conversioni.

```
ADMUX = 0b00000000; // Vcc (3.3V) = voltage reference
ADCSRA = 0b10000110; //Prescaler clock ADC = 64
```

Il registro ADMUX è quello che controlla la tensione di riferimento degli ADC.

Ponendo i bit REFS1 (settimo bit) e REFS0 (sesto bit) a 0 si imposta la tensione di riferimento a VCC.

ADCSRA è il registro che regola il prescaler degli ADC.

I bit meno significativi (ADPS2, ADPS1 e ADPS0) permettono il settaggio del prescaler.

Ponendo ADPS2 e ADPS1 a 1 mentre ADPS0 a 0 si setta un prescaler di 64.

Il bit più significativo si chiama ADEN e serve da “ADC enable”, ponendolo uguale a 1 si attiva l’ADC.

Nel caso ADEN fosse uguale a 0 non sarebbe possibile leggere la tensione poiché l’ADC sarebbe disattivato.

Una volta cambiato il software è stato ripetuto lo stesso test cercando di eguagliare le condizioni del primo.

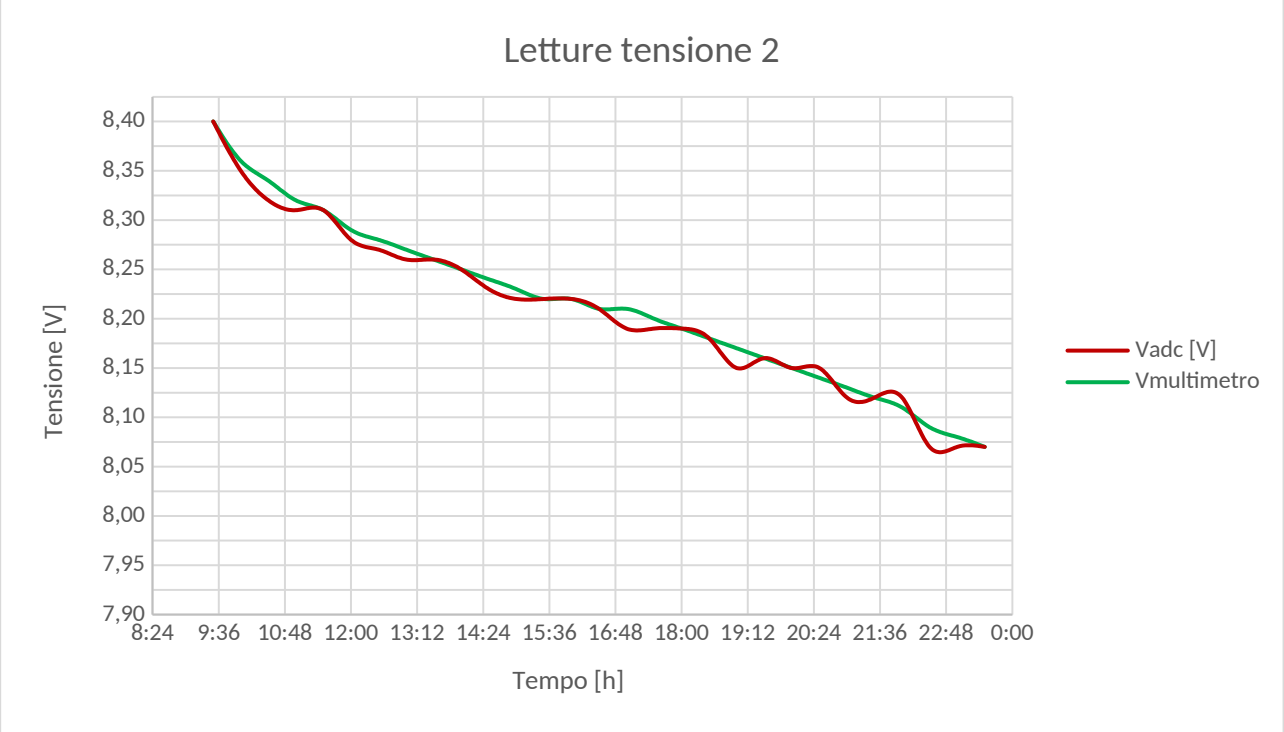
La metodologia di ottenimento del valore di tensione non è stata variata, come nel primo test la media si effettua su quattro campioni di tensione acquisiti a intervalli regolari di dieci secondi.

Come si vede dal grafico gli errori di lettura persistono ma sono notevolmente ridotti e la linea rossa riproduce più fedelmente la linea verde.

Rispetto al test precedente sono stati migliorati alcuni valori.

L’errore massimo misurato è di 10mV mentre sul periodo totale l’errore di tensione è di 7mV.

Con le modifiche apportate al software è stato possibile limitare l’errore nella lettura e l’errore totale che il sistema compie nella conversione della tensione.



Letture tensione 2

Vadc [V]	Vmultimetro [V]	Tempo [h]	errore [V]	errore [%]
8,40	8,40	9:30	0,000	0,000
8,35	8,36	10:00	0,010	0,120
8,32	8,34	10:30	0,020	0,240
8,31	8,32	11:00	0,010	0,120
8,31	8,31	11:30	0,000	0,000
8,28	8,29	12:00	0,010	0,121
8,27	8,28	12:30	0,010	0,121
8,26	8,27	13:00	0,010	0,121
8,26	8,26	13:30	0,000	0,000
8,25	8,25	14:00	0,000	0,000
8,23	8,24	14:30	0,010	0,121
8,22	8,23	15:00	0,010	0,122
8,22	8,22	15:30	0,000	0,000
8,22	8,22	16:00	0,000	0,000
8,21	8,21	16:30	0,000	0,000
8,19	8,21	17:00	0,020	0,244
8,19	8,20	17:30	0,010	0,122
8,19	8,19	18:00	0,000	0,000
8,18	8,18	18:30	0,000	0,000
8,15	8,17	19:00	0,020	0,245
8,16	8,16	19:30	0,000	0,000
8,15	8,15	20:00	0,000	0,000
8,15	8,14	20:30	0,010	0,123
8,12	8,13	21:00	0,010	0,123
8,12	8,12	21:30	0,000	0,000
8,12	8,11	22:00	0,010	0,123
8,07	8,09	22:30	0,020	0,247
8,07	8,08	23:00	0,010	0,124
8,07	8,07	23:30	0,000	0,000
		Totale	0,007	0,087

Per cercare di migliorare ulteriormente i risultati derivanti dalla lettura della tensione della batteria è stato effettuato un terzo test.

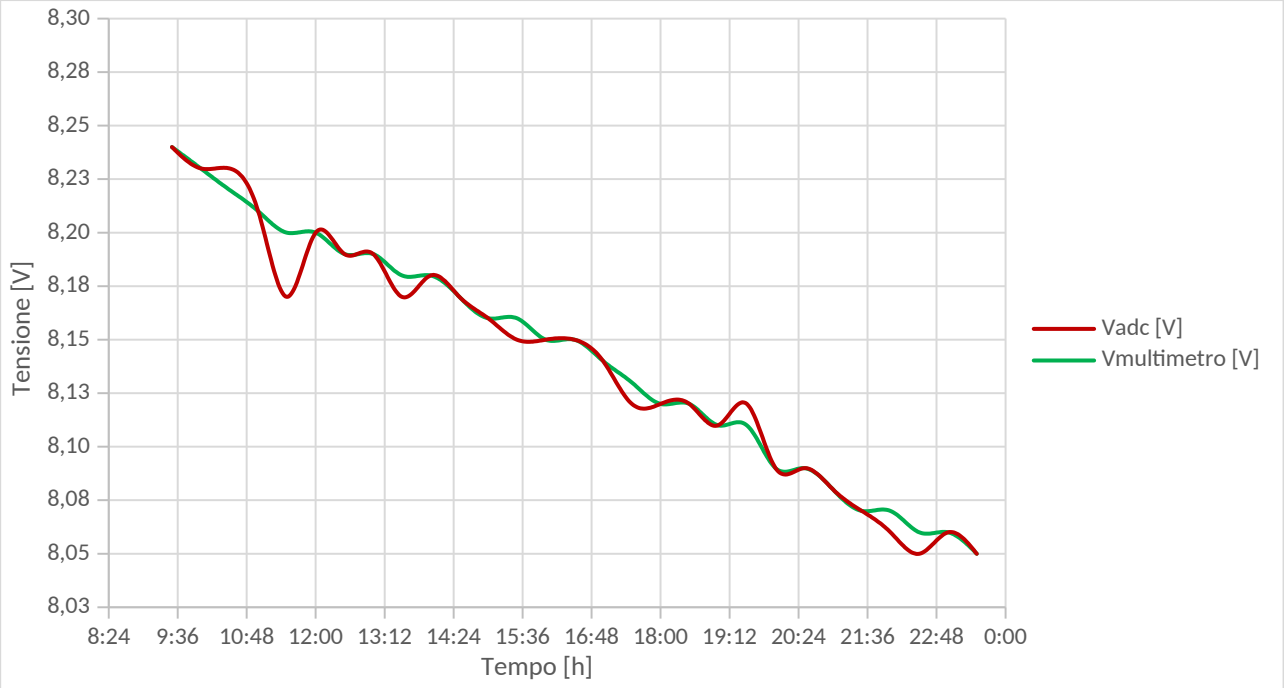
Rispetto ai due algoritmi precedenti è stato modificato il numero di campioni acquisiti per ottenere la media.

In questo test sono stati acquisiti dieci valori contro i quattro del primo e del secondo test.

Tramite una funzione appositamente creata il sistema riconosce ed elimina il valore maggiore e minore presente all'interno della serie.

Di conseguenza la media finale risulterà essere composta da otto valori.

Sono stati scelti dieci valori perché un numero più alto avrebbe richiesto una quantità di memoria superiore a quella disponibile.



La batteria utilizzata per il terzo test è diversa da quella del primo e del secondo.

A causa di un mal funzionamento la batteria, durante la fase di ricarica, ha preso fuoco ed è stato necessario sostituirla.

La seconda batteria è stata ricaricata prima dell'inizio del terzo test ma non ha raggiunto la tensione massima.

Letture tensione 3				
Vadc [V]	**Vmultimetro [V]**	**Tempo [h]**	**errore [V]**	**errore [%]**
8,24	8,24	9:30	0,000	0,000
8,23	8,23	10:00	0,000	0,000
8,23	8,22	10:30	0,010	0,122
8,21	8,21	11:00	0,000	0,000
8,17	8,20	11:30	0,030	0,366
8,20	8,20	12:00	0,000	0,000
8,19	8,19	12:30	0,000	0,000
8,19	8,19	13:00	0,000	0,000
8,17	8,18	13:30	0,010	0,122
8,18	8,18	14:00	0,000	0,000
8,17	8,17	14:30	0,000	0,000
8,16	8,16	15:00	0,000	0,000
8,15	8,16	15:30	0,010	0,123
8,15	8,15	16:00	0,000	0,000
8,15	8,15	16:30	0,000	0,000
8,14	8,14	17:00	0,000	0,000
8,12	8,13	17:30	0,010	0,123
8,12	8,12	18:00	0,000	0,000
8,12	8,12	18:30	0,000	0,000
8,11	8,11	19:00	0,000	0,000

8,12	8,11	19:30	0,010	0,123
8,09	8,09	20:00	0,000	0,000
8,09	8,09	20:30	0,000	0,000
8,08	8,08	21:00	0,000	0,000
8,07	8,07	21:30	0,000	0,000
8,06	8,07	22:00	0,010	0,124
8,05	8,06	22:30	0,010	0,124
8,06	8,06	23:00	0,000	0,000
8,05	8,05	23:30	0,000	0,000
		Totale	0,004	0,044

Rispetto ai due test precedenti le letture di tensione effettuate risultano più precise.

Nei primi test il risultato migliore ottenuto consisteva in un errore massimo di 10mV e di un errore sul periodo totale di 7mV.

In quest'ultimo test l'errore massimo è stato di 30mV mentre l'errore sul periodo totale di soli 4mV.

Di conseguenza il terzo algoritmo risulta essere il più accurato.

Si precisa che durante l'uso di tutti e tre gli algoritmi sono state rilevate alcune oscillazioni del valore letto della tensione.

Queste oscillazioni avvengono sporadicamente e sono state rilevate solamente una volta durante la fase di acquisizione della tensione.

Di seguito si riporta la funzione utilizzata per calcolare la media durante il terzo test.

```
float risultato(float vettore[], byte indice){
  volatile float v_max = 0, v_min = 0, somma = 0;
  for(int j = 0; j < indice; j++){
    if(vettore[j] > v_max) v_max = vettore[j];
    else if(vettore[j] < v_min) v_min = vettore[j];
    if(j == 0) v_min = v_max;
    somma += vettore[j];
  }
  return (somma - v_max - v_min) / (NumeroCampioni - 2.00);
}
```

Questa funzione è stata creata per rendere automatico tutto il processo di acquisizione e calcolo della tensione.

Se viene cambiato il numero di campioni da acquisire contenuto all'interno della variabile "NumeroCampioni" il sistema in automatico regola tutto il processo.

Table 22-6. ADC Characteristics, Single Ended Channels. –40°C to +125°C

Parameter	Condition	Symbol	Min	Typ	Max	Units
Resolution	Single-ended conversion			10		Bits
Absolute accuracy (Including	Single-ended conversion V_{REF} = 4V, V_{CC} = 4V, ADC clock = 200kHz			2.0	4.0	LSB

Come si può vedere dalla tabella del datasheet la risoluzione per l'ADC utilizzato in modalità "single endend" è di 10 bit.

Se l'ADC avesse un errore di un quanto in ogni lettura si avrebbe un'imprecisione di $\pm\frac{3.3\,V}{2^{10}} = \pm 3.2226\,mV$.

Dato che la precisione tipica è di ± 2 LSB si ha un errore in ogni lettura di $\pm 6.4453\,mV$, nel caso peggiore con l'errore massimo di ± 4 LSB si ottiene una precisione di $\pm 12.8906\,mV$.

Guardano i valori ottenuti con la terza versione dell'algoritmo di acquisizione si nota che sono in linea con le caratteristiche del datasheet.

Di seguito si riportano le formule utilizzate per la conversione.

```
x = (4.20 * analogRead(cella1)) / 817.53479;
y = (8.40 * analogRead(vbat)) / 789.97765;
```

La variabile "x" è associata ai valori della prima cella mentre "y" viene associata alla tensione totale della batteria.

Di seguito si riportano i calcoli effettuati per arrivare a queste formule.

Dato che il procedimento è il medesimo si riporteranno solo i calcoli per la formula della tensione totale della batteria.

Conoscendo la tensione massima che la batteria può raggiungere (8,4V) il partitore avrà una tensione d'uscita di 2,548314V.

$$V_{out} = \frac{R_2}{R_1 + R_2} * V_{in} = \frac{27000}{27000 + 62000} * 8.4\,V = 2.548314\,V$$

Questa tensione in valore digitale risulta essere uguale a 789,97765.

$$Valore\ \ Analogico = \frac{Vin * 1023}{Vref} = \frac{2.548314\,V * 1023}{3.3\,V} = 789.97765$$

Di conseguenza quando in entrata ci saranno 8,4V il valore digitale in uscita sarà quello sopracitato.

Per calcolare la tensione totale basterà eseguire la formula inversa e si troverà la tensione di alimentazione del partitore in funzione della tensione d'uscita.

$$V_{bat} = \frac{8.40\,V * Valore\ \ Analogico}{789.97765}$$

Il microcontrollore arrotonderà i valori digitali ottenuti tramite le letture degli ADC e quelli presenti nelle formule.

Sono stati lasciate molte cifre decimali ed è stato evitato di arrotondare per permettere al microcontrollore di eseguire un arrotondamento uguale per tutti i valori digitali inseriti.

In questo modo si diminuisce l'errore nelle letture causato da un errato arrotondamento.

Per fare un esempio si suppone la tensione d'ingresso del partitore di 7.57V.

Utilizzando la prima formula si ottiene la tensione d'uscita dal partitore.

Si ricorda che la tensione letta nel caso migliore è soggetta ad un errore di $\pm 3.2226\,mV$.

$$V_{out} = \frac{R_2}{R_1 + R_2} * V_{\text{in}} = \frac{27000}{27000 + 62000} * 7.57\,V = 2.296517\,V$$

Con la seconda formula si ottiene il valore analogico corrispondente alla tensione letta dall'ADC.

$$Valore \quad Analogico = \frac{Vin * 1023}{Vref} = \frac{2.296517\,V * 1023}{3.3\,V} = 711.92027$$

L'ultima formula permette di risalire alla tensione originale.

$$V_{bat} = \frac{8.40\,V * Valore \quad Analogico}{789.97765} = \frac{8.40\,V * 711.92027}{789.97765} = 7.569999\,V$$

EEPROM salvataggio impostazioni

Il sistema prevede un menu impostazioni a cui l'utente può accedere per modificare le temporizzazioni presenti all'interno del programma.

L'utente può modificare il tempo in cui vengono acquisiti i campioni di tensione, l'intervallo di tempo in cui viene aggiornato il display e l'intervallo di tempo di salvataggio del numero di proiettili sparati nella EEPROM.

Per evitare che questi settaggi si perdano ogni volta che viene tolta l'alimentazione al circuito tutti i dati sono salvati a loro volta all'interno della EEPROM.

Nella EEPROM vengono salvati gli indici di riferimento di ogni temporizzazione.

L'indice è il numero compreso tra uno e tre che permette all'utente di selezionare le differenti temporizzazioni.

Per facilitare la scrittura del programma tutte le temporizzazioni sono state inserite tramite dei "define".

Successivamente sono stati creati tre array (uno per temporizzazione) e le variabili di tempo sono state inserite al loro interno.

Ogni opzione ha un proprio indice che viene modificato a seguito delle azioni compiute dall'utente.

Per esempio la variabile "indicesalvataggio" contiene un valore compreso tra uno e tre che permette di stabilire quale temporizzazione l'utente ha scelto.

```
long temposalvataggio[3] = {salvataggio1, salvataggio2, salvataggio3};
long tempodisplay[3] = {updatedisplay1, updatedisplay2, updatedisplay3};
long tempotensione[3] = {letturatensione1, letturatensione2, letturatensione3};
```

Questi indici possono essere modificati dall'utente solamente all'interno del menu impostazioni.

Una volta che l'utente ritorna nella schermata principale gli indici verranno salvati.

È importante ricordare che gli indici verranno salvati solamente se uno o più cambiano, nel caso l'utente accedesse al menu delle impostazioni solamente per visionare i tempi e poi ritornasse alla schermata principale senza modificare nulla questi indici non sarebbero salvati.

```
if((eepromsalvataggio != indicesalvataggio) || (eepromdisplay != indicedisplay) || (eepromtensione != indicetensione)){
  EEPROM.write(numerotensione, indicetensione);
  EEPROM.write(numerodisplay, indicedisplay);
  EEPROM.write(numerosalvataggio, indicesalvataggio);
  eepromsalvataggio = indicesalvataggio;
  eepromdisplay = indicedisplay;
  eepromtensione = indicetensione;
}
```

Gli indici vengono caricati all'interno di variabili presenti nella RAM del microcontrollore durante la fase di boot ogni volta che il sistema viene alimentato.

EEPROM salvataggio partite

Il sistema memorizza nella EEPROM il numero di colpi sparati nelle ultime cinque partite.

In automatico quando il sistema rileva che una nuova partita è in corso cancella i dati della quinta partita e sposta di una posizione gli altri in modo tale da liberare la prima posizione per la partita in corso.

Una volta che una nuova partita è stata rilevata il sistema aggiorna i dati sulla EEPROM con un intervallo di tempo uguale a quello selezionato nel menu impostazioni dall'utente.

Il sistema salva i dati nella EEPROM solamente se il numero di colpi è diverso da quello dell'ultimo salvataggio, in caso contrario il sistema non sovrascrive i dati e aspetterà il salvataggio successivo.

L'utente può decidere di salvare i dati indipendentemente dal fatto che sia passato l'intervallo di tempo da lui fissato.

Per eseguire il salvataggio per prima cosa bisogna entrare nella schermata dove vengono mostrate le ultime partite.

Successivamente si deve premere il pulsante "avanti" per più di due secondi.

Una volta che il tempo sarà passato il sistema salverà i dati solamente se il numero di proiettili è differente da quello dell'ultimo salvataggio.

Nel caso in cui il sistema salvi i dati comparirà una schermata che conferma l'avvenuto salvataggio, in caso contrario il sistema tornerà nella schermata principale per indicare che il numero salvato è già il dato più recente.

Nel caso l'utente smettesse di premere il pulsante prima che siano passati i due secondi il sistema azzererà il conteggio.

A questo punto l'utente può premere nuovamente il pulsante per salvare i dati.

Si precisa che se l'utente rilascia il pulsante prima dei due secondi non verrà mostrato nessun segnale d'avvertimento, il programma rimane nella schermata senza mostrare nulla.

```
if(lastpallini != pallini){
  if(nuovapartita == false){
    nuovapartita = true;
    partite[4] = partite[3];
    partite[3] = partite[2];
    partite[2] = partite[1];
    partite[1] = partite[0];
    EEPROM.put(gioco5, partite[4]);
    EEPROM.put(gioco4, partite[3]);
    EEPROM.put(gioco3, partite[2]);
    EEPROM.put(gioco2, partite[1]);
  }
  partite[0] = pallini;
  lastpallini = pallini;
  EEPROM.put(gioco1, partite[0]);
```

Rilevazione proiettili

La rilevazione dei proiettili è affidata a un sistema che comprende un diodo IR, un fotodiodo, un comparatore LM393 e un potenziometro che permette di regolare la soglia di sensibilità del sistema.

Come sensore è stato utilizzato un fotodiodo perché risulta essere uno dei più veloci.

Il fotodiodo viene utilizzato in polarizzazione inversa, come uno zener, perché in questo modo permette il passaggio di corrente quando è esposto alla luce ed entra in interdizione una volta che viene oscurato.

il comparatore LM393 presenta un tempo di risposta di pochi µs, di conseguenza risulta ideale per la corretta acquisizione di segnali veloci come quelli che si cercano di acquisire.

Per essere sicuri che il microcontrollore possa acquisire tutti i segnali provenienti dal comparatore in qualsiasi momento ed in qualsiasi punto del programma è stato utilizzato un interrupt.

Per settare correttamente l'interrupt in modo tale che legga solamente i segnali sui fronti di discesa sono state scritte le seguenti righe di codice.

```
GIMSK = 0b01000000; // interrupt 0
MCUCR = 0b00000010; // falling edge
sei(); // abilita gli interrupt
```

Il registro GIMSK permette di decidere quali pin di interrupt attivare, in questo caso settando a 1 il bit INT0 si inizializza l'interrupt 0.

Il secondo registro utilizzato, MCUCR, serve per settare la modalità di funzionamento degli interrupt.

Dato che è stato deciso di leggere i segnali sui fronti di discesa è necessario settare il bit ISC01 a 1 e il bit ISC00 a 0.

La funzione "sei()" abilita gli interrupt.

Tutte queste informazioni sono state prese dal datasheet del microcontrollore e dal sito della Microchip.

Ogni volta che viene triggerato l'interrupt il programma si interrompe e viene eseguita la routine di interrupt.

In questo caso all'interno della routine si aumenta di uno la variabile che mantiene in memoria il numero di colpi sparati e si modificano alcune variabili necessarie per il corretto funzionamento del sistema.

```
ISR(EXT_INT0_vect){
  // Per AtTiny44/84  EXT_INT0_vect
  // Per AtTiny45/85  INT0_vect
  pallini++;
}
```

Il nome del bit di flag che viene posto a 1 ogni volta che viene scatenato l'interrupt è "EXT_INT0_vect" e si possono trovare tutte le caratteristiche sul sito della Microchip.

Per impostare la corretta tensione di riferimento sul pin invertente dell'operazionale sono state effettuate numerose prove.

Il trimmer da 50KΩ utilizzato è un trimmer multigiri.

È stato utilizzato questo componente perché permette di regolare in modo preciso la tensione, un potenziometro normale non permetterebbe una regolazione accurata.

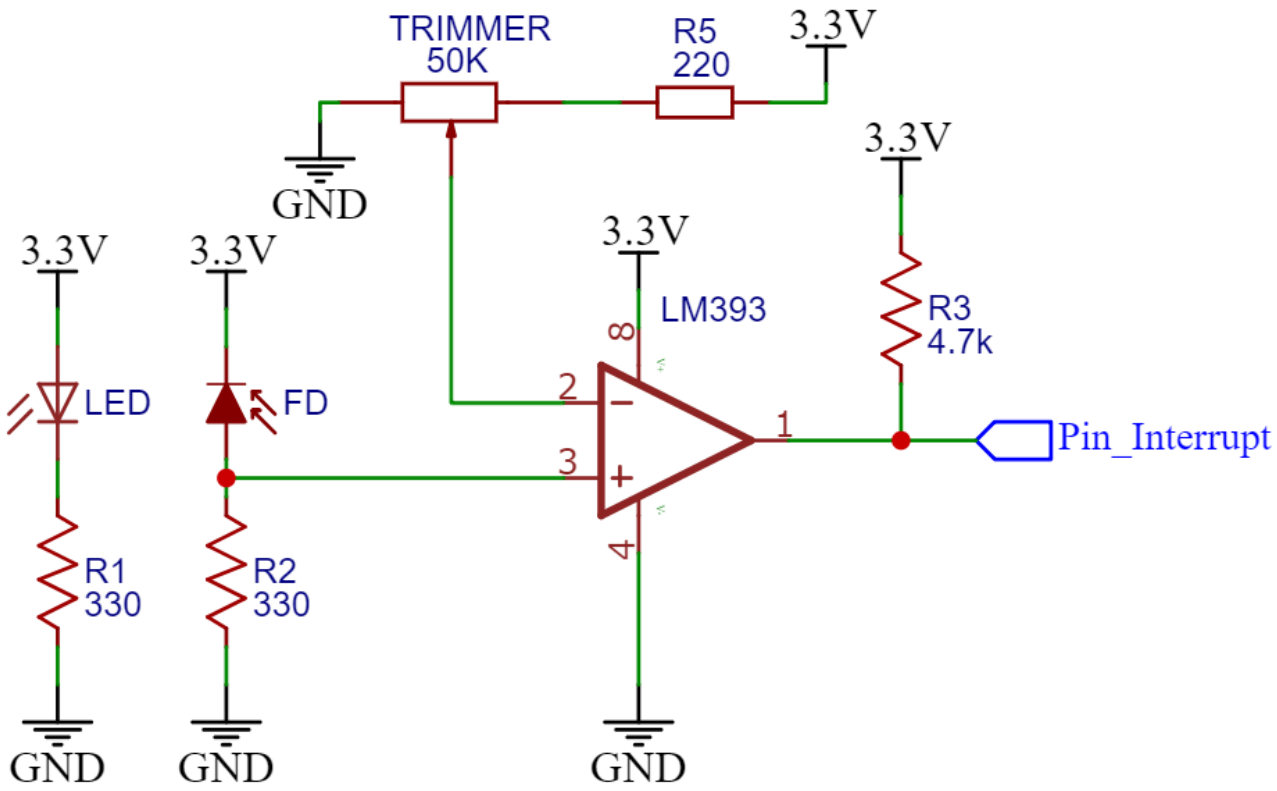
Dopo vari test è stato riscontrato che la tensione alla quale si ottiene la migliore precisione è di circa 410mV.

La tensione presente sul pin non invertente in assenza di proiettili che interrompono il fascio luminoso è di circa 920mV.

Di conseguenza la tensione sull'ingresso invertente risulta essere poco meno della metà della tensione che cade sulla resistenza R_2.

Di seguito si riporta lo schema del sistema di rilevamento dei proiettili.

Si ricorda che questo schema differisce dallo schema reale poiché è usato a scopo dimostrativo.



Display e menu

Per comunicare col display OLED è stato utilizzato il protocollo I2C.

Di conseguenza è stato stabilito un Master (ATtiny84) ed uno Slave (display OLED).

Questo protocollo è vantaggioso perché permette di interfacciare il display col microcontrollore utilizzando solamente due cavi.

Se si fosse utilizzata un'interfaccia parallela il numero di connessioni sarebbe stato maggiore.

È stato necessario inserire nel circuito due resistenze di pull-up da 10kΩ per mantenere allo stato logico alto le linee SDA e SCL.

SDA (Serial DAta) è il cavo che trasmette i dati da Master a Slave o viceversa.

SCL (Serial CLock) è il cavo che porta il segnale di clock creato dal Master, per questo motivo l'I2C è considerata una comunicazione sincrona.

È stata utilizzata una libreria apposita che permette di semplificare il codice poiché contiene al suo interno tutti i comandi che si possono utilizzare per comunicare con il display.

Tramite l'utilizzo di determinate funzioni è possibile inizializzare il display, scrivere, cancellare e disegnare.

Lo scambio di informazioni avviene solamente quando il Master invia o richiede dei dati ad un determinato Slave.

Con questa tipologia di comunicazione si potrebbero utilizzare fino a 127 tra sensori e display.

Come in ogni comunicazione seriale è presente un bit di start e un bit di stop.

Il menu impostazioni permette all'utente di modificare il tempo di lettura della tensione, il tempo di refresh del display e il tempo di salvataggio dei dati nella EEPROM.



Il numero a destra della scritta "Impostazioni" permette di selezionare l'impostazione desiderata.

Al numero uno è associata l'impostazione "Tensione", al due "Display" e al tre "EEPROM".

Premendo il pulsante "avanti" questo numero verrà incrementato e, una volta raggiunto il numero tre, verrà riportato uguale a uno.

Quando l'indice è uguale all'impostazione che si vuole modificare si preme il pulsante "conferma" e il valore a destra dell'impostazione selezionata cambierà.

Si riporta di seguito la parte di codice relativa a questa sezione del programma e successivamente si descrive come funziona il tutto.

```
if(digitalRead(avanti) == HIGH){
  if(m < 3) m++;
  else{
    m = 1;
  }
  delay(100);
}
if(digitalRead(10) == HIGH){
  switch(m){
    case 1:
           if(indicetensione < 3) indicetensione++;
           else{
             indicetensione = 1;
             oled.clearDisplay();
           }
           delay(100);
           break;
   case 2:
           if(indicedisplay < 3) indicedisplay++;
           else{
             indicedisplay = 1;
             oled.clearDisplay();
           }
           delay(100);
           break;
   case 3:
           if(indicesalvataggio < 3) indicesalvataggio++;
           else{
             indicesalvataggio = 1;
             oled.clearDisplay();
           }
           delay(100);
           break;
    }
  }
}
```

```
#define letturatensione1 6000  // 1
#define letturatensione2 18000 // 2
#define letturatensione3 30000 // 3
```

```
long temposalvataggio[3] = {salvataggio1, salvataggio2, salvataggio3};
long tempodisplay[3] = {updatedisplay1, updatedisplay2, updatedisplay3};
long tempotensione[3] = {letturatensione1, letturatensione2, letturatensione3};
```

Nel caso il numero a destra di “impostazioni” fosse uguale a uno (nel programma è rappresentato dalla variabile “m”) l’impostazione “Tensione” verrebbe selezionata.

Nel momento in cui viene premuto il tasto “conferma” il programma incrementa di uno la variabile “indicetensione”.

Questa variabile viene usata per spostarsi all’interno dell’array “tempotensione” che contiene i tre valori che l’utente può selezionare.

Per esempio se “indicetensione” avesse un valore uguale a due verrebbe selezionato “letturatensione2” all’interno dell’array che in questo caso corrisponde a 18000 ms (18 secondi).

Per evitare fraintendimenti si specifica che all’interno del programma, per ottenere i valori corretti, viene sottratto uno al valore dei tre indici (“indicetensione”, “indicedisplay” e “indicesalvataggio”) poiché il primo posto all’interno degli array è identificato col numero zero e non uno.

Si riporta di seguito un esempio.

```
if(millis() - t > tempotensione[eepromtensione - 1])
```

Una volta che il sistema salva gli indici “eepromtensione” e “indicetensione” si equivalgono.

Ogni volta che un indice cambia il nuovo valore verrà mostrato sul display OLED.

Le varie temporizzazioni sono salvate all’interno di tre array che facilitano la selezione e il salvataggio dei valori.

Una volta che l’utente ha cambiato le impostazioni che desidera per salvare dovrà tornare alla schermata principale.

Conclusioni

Il sistema di conteggio dei proiettili funziona correttamente.

Il comparatore è sufficientemente veloce per rilevare il passaggio di un proiettile.

I dati che vengono salvati nella EEPROM rimangono memorizzati correttamente e sono visualizzabili anche dopo aver tolto l’alimentazione.

La lettura della tensione è abbastanza accurata per il fine di questo circuito.

Sarebbe stato opportuno inserire nel circuito un ADC esterno con un numero maggiore di bit così da avere un errore più piccolo rispetto alla minima tensione letta.

In commercio esistono ADC, per esempio l’ADS1115, che permettono un interfacciamento con protocollo I2C.

Dato che anche il display funziona con lo stesso protocollo non si avrebbe la necessità di creare ulteriori bus per i dati.

Il microcontrollore utilizzato presenta tutte le caratteristiche necessarie per il corretto funzionamento del circuito ma possiede una scarsa quantità di memoria.

Si potrebbe ovviare a questo problema utilizzando un microcontrollore differente oppure una memoria esterna.

Per esempio si potrebbe utilizzare un ATmega328P che dispone di una memoria da 32KB e un numero maggiore di pin I/O.

Uno degli obiettivi di questa scheda era quello di creare un sistema funzionale che permettesse di fare tutte le cose che fanno i sistemi simili in commercio.

Alcune aziende vendono prodotti simili ma costano centinaia di euro.

Questo progetto, pur non essendo perfetto, permette di contare i proiettili e di tenere sotto controllo la tensione in modo accettabile per un costo di circa tredici euro.